



Hochschule Ravensburg-Weingarten
University of Applied Sciences

Embedded Computing Lab Task 4

Author: Prachan Chandrashekar

Matrikel-Number: 27143773

Contents

Task 1	4
Task 2A	8
Task 2B	8
Objective:	8
Code:	9
Steps:	11
Expected Result:	11
Actual Result:	11
Observation:	11
Task 2C	11
Objective:	11
Code:	11
Expected Result:	13
Actual Result:	13
Observation:	13
Task 2D	14
Task 2E	14
Task 2E.1	14
Objective:	15
Code Snippet:	15
Expected Result:	16
Actual Result:	16
Observation:	16
Task 2E.2	17
Objective:	17
Theory:	17
Code:	17
Expected Result:	20
Actual Result:	20
Output:	21
Observation:	21
Task 2F	21
Objective:	21
Theory:	21
Code:	22
Expected Result:	25
Actual Result:	25
Observation:	25
Task 2G	25
Objective:	25
Code:	25

Output:	26
Task 3.....	26
Objective:	27
Implementation Details	27
Create a python script to move the robot forward till obstacle is at distance around 10cm from it.	27
Approach:.....	27
Challenges faced:	28
Modify webserver.js to trigger or execute the python script from the checkbox in index.html for any changes.	28
Approach:.....	28
Proposed Solution:.....	29
The data from the robot such as distance from obstacle, distance traveled by each wheel (Number of encoder ticks) has to be displayed after the script stops.	30
Approach:.....	30
Proposed solution:	30
Create a second python script to move robot backward by around 20cm and turn random angle.	31
Approach:.....	31
Challenge Faced:	31
Proposed solution:	31
After the second script call the first script to move forward till obstacle.....	32
Approach:.....	32
Proposed Solutions:	32
Create another Checkbox in index.html, that will execute a second python script followed by first python script.....	33
Approach:.....	33
Create a Third Checkbox in index.html, The third check box when checked will do following:	33
Approach:.....	33
Proposed Solution:.....	34
Results:	35
Task 4.....	36
Objective:	36
Implementation Details	37
C Program: sr04_client.c	37
Python Sensor Server: soc_server.py	37
Python Motor Controller: motor_task4.py	37
Node.js Webserver and UI Integration.....	38
Results:	38

Task 1

- a) Connect the ET1 using the details given in the info pdf.
- b) Using the VNC Viewer and verify the image has the necessary programs on. Items to verify are:
 - Thonny (Python IDE)
 - C-Development Environment
 - Node.JS installation
 - Firefox

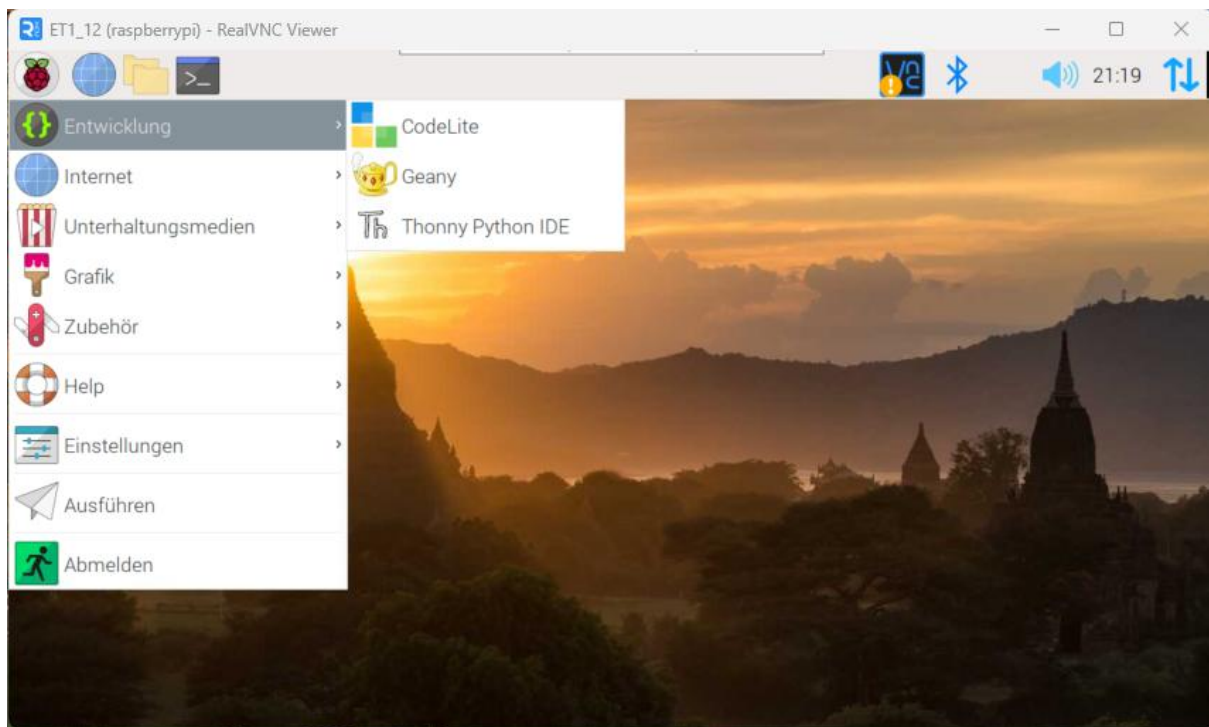


Figure 1: Connected to ET1(Raspberry Pi) using VNC Viewer

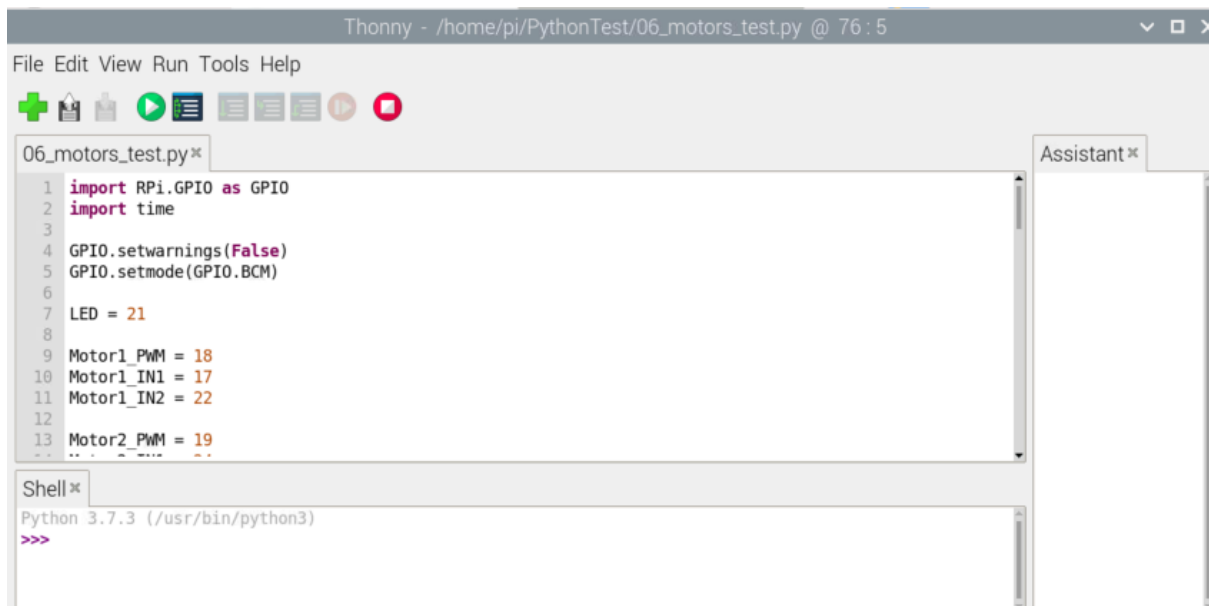


Figure 2: Thonny Python IDE

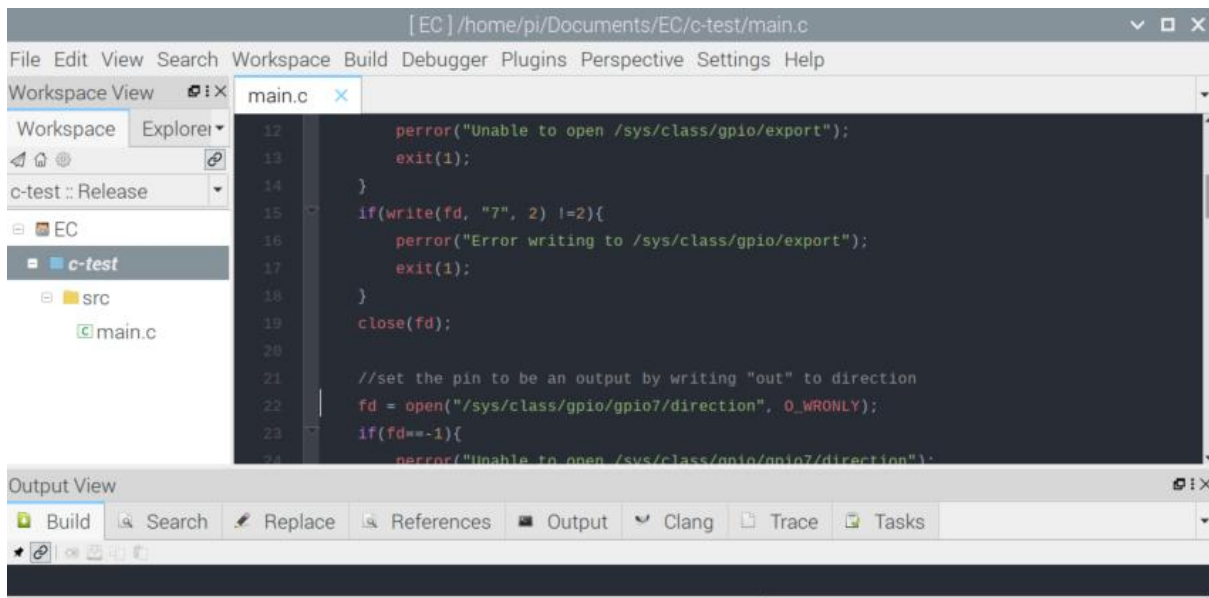


Figure 3: Codelite (C development Environment)

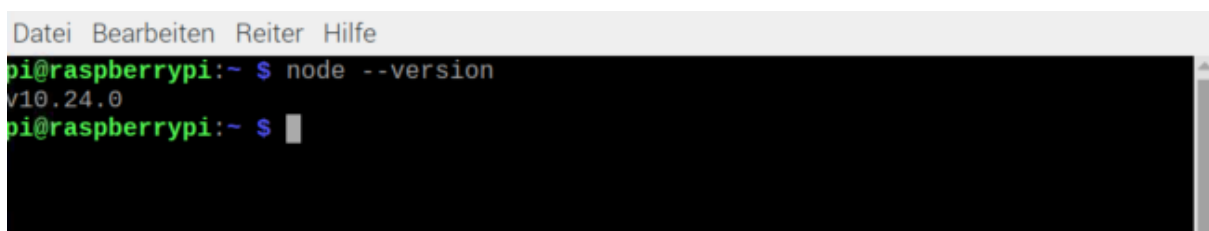


Figure 4: Node.js installed

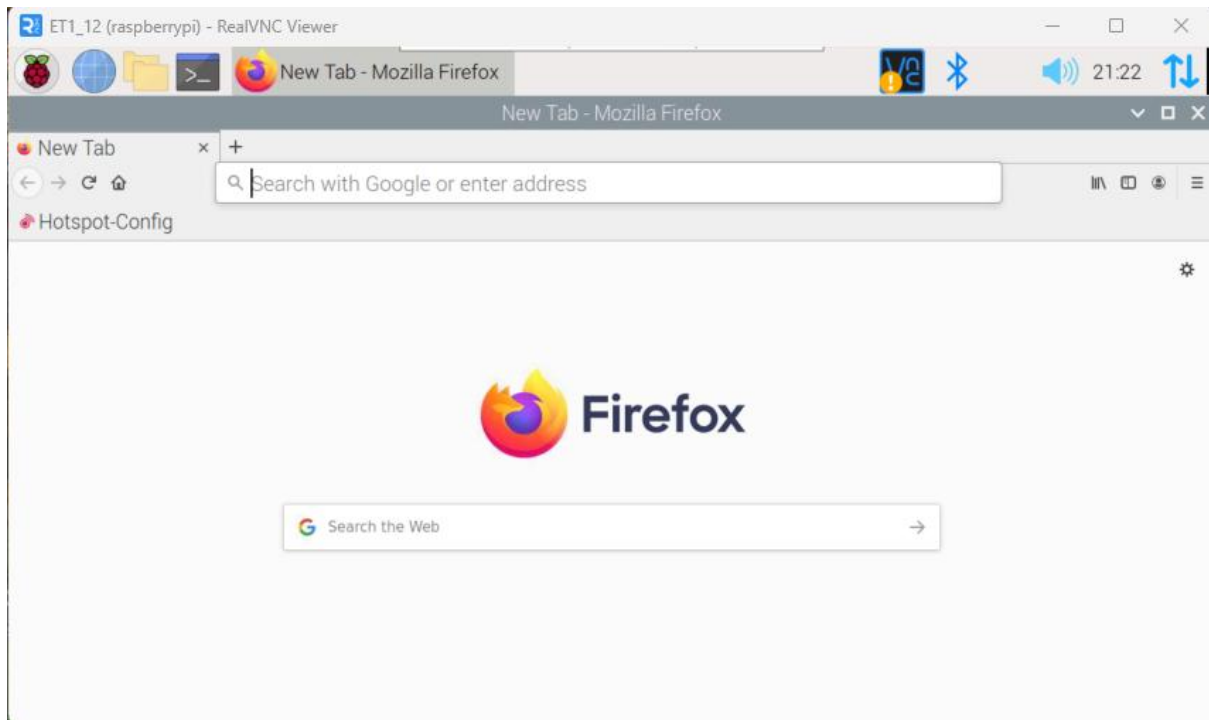


Figure 5: Firefox

c) Using the Thonny Python IDE find the test python scripts and evaluate what they do.

Sl.No	Test Script	Review Comment	Test
1	00_hello-world_and_count.py	This python script prints the hello world 7 times	Passed
2	01_blink.py	This python script will turn ON and OFF the LED at GPIO 7 every 1 second in output console it will print the number of blink cycles.	Passed
3	02_blink_pushbutton.py	This python script will turn OFF the LED at GPIO 7 when the push button at GPIO5 is pressed	Passed
4	03_blink_keyboard.py	This python script takes input from the keyboard, if we enter Character e it turns ON the LED at GPIO 7 Character a turn OFF the LED at GPIO 7. Character q exits the test script	Passed
5	05_distance_SR04.py	This Script gets the distance from the SR04 Ultrasonic distance sensor.	Passed

6	06_motors_test.py	The Script runs 2 motor forward and backward for 1 second each and stops for 0.5 seconds when in between the transaction. We are using the following Pins for the motor H bridge connections: Motor 1: PWM = GPIO18 IN1 = GPIO17 IN2 = GPIO22 Motor2 PWM = GPIO19 IN1 = GPIO24 IN2 = GPIO4	Passed
7	07_motors_pwm.py	The Script runs 2 motor with PWM frequency of 90Hz. Initially both the motors run at 40% duty cycle forward and then sleep for 5 seconds. Next the motor1 runs with 80% duty cycle forward and motor2 runs with 40% duty cycle backward, then sleep for 5 seconds. Then both PWM is set to 0% for 3 seconds. H bridge connections: Motor 1: PWM = GPIO18 IN1 = GPIO17 IN2 = GPIO22 Motor2 PWM = GPIO19 IN1 = GPIO24 IN2 = GPIO5	Passed
8	08_cyrcl_drive.py	The Script runs 2 motors with PWM frequency of 90Hz. This script tries to rotate ET1 in circle. To achieve this the motor1 run at 80% duty cycle forward and motor2 runs with 20% duty cycle forward, then sleep for 30 seconds. Then both PWM is set to 0% duty cycle for 2 seconds. H bridge connections: Motor 1: PWM = GPIO18 IN1 = GPIO17 IN2 = GPIO22 Motor2 PWM = GPIO19 IN1 = GPIO24 IN2 = GPIO6	Passed

		The Motors are turned according to keyboard inputs. The key board input and functionality are as given below: z: Motors run forward with 30% duty cycle y: Motors run forward with 95% duty cycle b: Motors run backward with 50% duty cycle g: Motor1 runs 30% and motor2 runs 50% duty cycle forward. this is expected to turn left. h: Motor1 runs 70% and motor2 runs 20% duty cycle forward. This is expected to turn right. s: Motors are set to 0% duty cycle. q: quit the test.	
9	09_motors_keyboard.py		Passed
10	10_CNY70_test.py	This Test just checks line is black or white and prints it.	Passed
11	11_distance_sensor_test.py	This test checks and prints if the sensor is Open or closed.	Passed
12	12_distance_sensors_interrupt.py	This test checks and prints if the sensor is Open or closed based on the event.	Passed

d) Review the Tinkercad Simulation of the H-Bridge that you did and find the correct output pins on the ET1 to use in order to get a similar behaviour.

Motor 1	PWM	GPIO18	D9
	IN1	GPIO17	D8
	IN2	GPIO22	D7
Motor2	PWM	GPIO19	D3
	IN1	GPIO24	D4
	IN2	GPIO4	D5

Task 2A

Using Python, query the pushbutton and use it to switch LED on and off to test the functionality of the Button.

Using the python test script 02_blink_pushbutton.py the functionality of the push button is tested

Task 2B

Using the pushbutton, cause the robot to rotate on the spot for 10 seconds, using the left wheel and then another 10 seconds using the right wheel.

Objective:

To rotate the robot on spot using left wheel (clock wise) and right wheel (anti-clock wise) respectively on click of the push button.

Code:

Below is the code used to perform the task of rotation on spot.

```
import RPi.GPIO as GPIO
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

# Pin definitions
LED = 7          # Pin for LED
Taster = 5       # Pin for pushbutton

# Motor 1 (Left) pin definitions
Motor1_PWM = 18  # PWM pin for Motor 1
Motor1_IN1 = 17  # Input pin 1 for Motor 1
Motor1_IN2 = 22  # Input pin 2 for Motor 1

# Motor 2 (Right) pin definitions
Motor2_PWM = 19  # PWM pin for Motor 2
Motor2_IN1 = 24  # Input pin 1 for Motor 2
Motor2_IN2 = 4   # Input pin 2 for Motor 2

duty_cycle = 0   # Initialize PWM duty cycle

# GPIO setup for LED and pushbutton
GPIO.setup(LED, GPIO.OUT)          # Set LED pin as output
GPIO.setup(Taster, GPIO.IN, pull_up_down=GPIO.PUD_UP) # Set pushbutton pin as input with pull-up resistor

# Function to setup Motor 1 (Left) control
def M1_setup():
    GPIO.setup(Motor1_IN1, GPIO.OUT) # Set Motor1_IN1 as output for direction control
    GPIO.setup(Motor1_IN2, GPIO.OUT) # Set Motor1_IN2 as output for direction control
    GPIO.setup(Motor1_PWM, GPIO.OUT) # Set Motor1_PWM pin as output for speed control
    PWM_1 = GPIO.PWM(Motor1_PWM, 90) # Initialize PWM with 90Hz frequency
    return PWM_1

# Function to setup Motor 2 (Right) control
def M2_setup():
    GPIO.setup(Motor2_IN1, GPIO.OUT) # Set Motor2_IN1 as output for direction control
    GPIO.setup(Motor2_IN2, GPIO.OUT) # Set Motor2_IN2 as output for direction control
    GPIO.setup(Motor2_PWM, GPIO.OUT) # Set Motor2_PWM pin as output for speed control
    PWM_2 = GPIO.PWM(Motor2_PWM, 90) # Initialize PWM with 90Hz frequency
```

```

return PWM_2

# Function to rotate Motor 1 forward
def M1_forward():
    GPIO.output(Motor1_IN2, GPIO.LOW) # Set Motor1_IN2 LOW for forward rotation
    GPIO.output(Motor1_IN1, GPIO.HIGH) # Set Motor1_IN1 HIGH for forward rotation

# Function to rotate Motor 1 backward
def M1_backward():
    GPIO.output(Motor1_IN1, GPIO.LOW) # Set Motor1_IN1 LOW for backward rotation
    GPIO.output(Motor1_IN2, GPIO.HIGH) # Set Motor1_IN2 HIGH for backward rotation

# Function to rotate Motor 2 forward
def M2_forward():
    GPIO.output(Motor2_IN2, GPIO.LOW) # Set Motor2_IN2 LOW for forward rotation
    GPIO.output(Motor2_IN1, GPIO.HIGH) # Set Motor2_IN1 HIGH for forward rotation

# Function to rotate Motor 2 backward
def M2_backward():
    GPIO.output(Motor2_IN1, GPIO.LOW) # Set Motor2_IN1 LOW for backward rotation
    GPIO.output(Motor2_IN2, GPIO.HIGH) # Set Motor2_IN2 HIGH for backward rotation

# Initialize PWM for both motors and set initial duty cycle to 0
PWM_1 = M1_setup()
PWM_1.start(0)           # Start PWM for Motor 1 with 0% duty cycle
PWM_2 = M2_setup()
PWM_2.start(0)           # Start PWM for Motor 2 with 0% duty cycle

while (1):
    if GPIO.input(Taster) == GPIO.LOW:    # Check if pushbutton is pressed
        #Rotate robot using left wheel for 10 seconds
        PWM_1.ChangeDutyCycle(80)         # Set duty cycle to 80% for left motor (Motor 1)
        M1_forward()                      # Rotate left motor forward
        time.sleep(10)                    # Wait for 10 seconds
        PWM_1.ChangeDutyCycle(0)          # Stop left motor by setting the duty cycle to 0

        #Rotate robot using right wheel for 10 seconds
        PWM_2.ChangeDutyCycle(80)         # Set duty cycle to 80% for right motor (Motor 2)
        M2_forward()                      # Rotate right motor forward
        time.sleep(10)                    # Wait for 10 seconds
        PWM_2.ChangeDutyCycle(0)          # Stop right motor by setting the duty cycle to 0

```

Steps:

1. First connect to the ET1 using VNC Viewer and create a new python file and copy or type the code given above.
2. Run the python code either using the thonny Python IDE or using the Terminal.
3. After running the code press the push button on ET1.

Expected Result:

On pressing the push button, the ET1 should first rotate clock wise for 10 seconds and then anti-clock wise for 10 seconds and stop.

Actual Result:

On pressing the push button is, the ET1 should first rotate clock wise for 10 seconds and then anti-clock wise for 10 seconds and stop.

Observation:

Even though the we have used the **same duty cycle and delay** the **amount of rotation of the ET1 is more** when made to rotate **using Left motor** (clockwise) than right motor.

Task 2C

Using the pushbutton, cause the robot to drive ahead with increasing speed and then decelerate and stop. Then drive the robot in exactly the same manner backwards.

Objective:

To drive the robot forward with increasing speed and the decelerate and stop. To drive the robot backward first with increasing speed then decelerate and stop.

Code:

Below is the code snippet used to perform the task.

```
# Function to move forward for given interval
def move_forward(PWM1, PWM2, dutycycle, interval):
    PWM1.ChangeDutyCycle(dutycycle)
    PWM2.ChangeDutyCycle(dutycycle)
    M1_forward()
    M2_forward()
    time.sleep(interval)

# Function to move backward for given interval
def move_backward(PWM1, PWM2, dutycycle, interval):
    PWM1.ChangeDutyCycle(dutycycle)
```

```

PWM2.ChangeDutyCycle(dutycycle)
M1_backward()
M2_backward()
time.sleep(interval)

# Callback function for left sensor detection to count ticks
def sensor_left(Channel):
    global left_sensor_tick_count
    left_sensor_tick_count += 1

# Callback function for right sensor detection to count ticks
def sensor_right(Channel):
    global right_sensor_tick_count
    right_sensor_tick_count += 1

# Function to print sensor tick data
def print_ticks(duty, direction):
    global left_sensor_tick_count, right_sensor_tick_count
    print(f"{direction} : Dutycycle: {duty} --> Left ticks: {left_sensor_tick_count}, Right ticks: {right_sensor_tick_count} ")

    left_sensor_tick_count = 0
    right_sensor_tick_count = 0

GPIO.add_event_detect(16, GPIO.RISING, callback = sensor_left)
GPIO.add_event_detect(23, GPIO.RISING, callback = sensor_right)

while (1):

    if GPIO.input(Taster) == GPIO.LOW:
        # Execute forward movement sequence with varying duty cycles
        move_forward(PWM_1, PWM_2, 30, 1)
        print_ticks(30, "Forward")
        move_forward(PWM_1, PWM_2, 40, 1)
        print_ticks(40, "Forward")
        move_forward(PWM_1, PWM_2, 60, 1)
        print_ticks(60, "Forward")
        move_forward(PWM_1, PWM_2, 80, 1)
        print_ticks(80, "Forward")
        move_forward(PWM_1, PWM_2, 60, 1)
        print_ticks(60, "Forward")
        move_forward(PWM_1, PWM_2, 40, 1)
        print_ticks(40, "Forward")
        move_forward(PWM_1, PWM_2, 30, 1)
        print_ticks(30, "Forward")
        move_forward(PWM_1, PWM_2, 0, 1)
        print_ticks(00, "Forward")
        time.sleep(5)

```

```

print("forward done")
PWM_1.ChangeDutyCycle(0)
PWM_2.ChangeDutyCycle(0)

# Execute backward movement sequence with varying duty cycles
move_backward(PWM_1, PWM_2, 30, 1)
print_ticks(30, "Backward")
move_backward(PWM_1, PWM_2, 40, 1)
print_ticks(40, "Backward")
move_backward(PWM_1, PWM_2, 60, 1)
print_ticks(60, "Backward")
move_backward(PWM_1, PWM_2, 80, 1)
print_ticks(80, "Backward")
move_backward(PWM_1, PWM_2, 60, 1)
print_ticks(60, "Backward")
move_backward(PWM_1, PWM_2, 40, 1)
print_ticks(40, "Backward")
move_backward(PWM_1, PWM_2, 30, 1)
print_ticks(30, "Backward")
move_backward(PWM_1, PWM_2, 0, 1)
print_ticks(00, "Backward")
time.sleep(5)

time.sleep(5)
print("backward done")
PWM_1.ChangeDutyCycle(0)
PWM_2.ChangeDutyCycle(0)

```

Expected Result:

On pressing the push button,

- The ET1 move forward in straight direction with acceleration and then decelerate to come to stop.
- Start moving backward in straight line with acceleration and then decelerate to come to stop.
- It should come back to the same place as started.

Actual Result:

- The ET1 is not moving forward in straight direction, it is steering towards right and in some runs even make a circle.
- The ET1 is not moving backward in straight line, it is steering towards right and in most runs it is going in circles.
- ET1 doesn't come back to starting position.

Observation:

Below is the output from one of the runs

Forward : DutyCycle: 30 --> Left ticks: 40, Right ticks: 32
Forward : DutyCycle: 40 --> Left ticks: 56, Right ticks: 46
Forward : DutyCycle: 60 --> Left ticks: 64, Right ticks: 58
Forward : DutyCycle: 80 --> Left ticks: 69, Right ticks: 65
Forward : DutyCycle: 60 --> Left ticks: 65, Right ticks: 59
Forward : DutyCycle: 40 --> Left ticks: 58, Right ticks: 50
Forward : DutyCycle: 30 --> Left ticks: 50, Right ticks: 39
Forward : DutyCycle: 0 --> Left ticks: 8, Right ticks: 2
forward done

Backward : DutyCycle: 30 --> Left ticks: 41, Right ticks: 25
Backward : DutyCycle: 40 --> Left ticks: 56, Right ticks: 44
Backward : DutyCycle: 60 --> Left ticks: 65, Right ticks: 56
Backward : DutyCycle: 80 --> Left ticks: 70, Right ticks: 63
Backward : DutyCycle: 60 --> Left ticks: 66, Right ticks: 57
Backward : DutyCycle: 40 --> Left ticks: 59, Right ticks: 46
Backward : DutyCycle: 30 --> Left ticks: 51, Right ticks: 37
Backward : DutyCycle: 0 --> Left ticks: 7, Right ticks: 4
backward done

- The left motor consistently registers higher tick counts than the right motor at all duty cycles, both in the forward and backward directions.
- Because of these higher ticks at the left motor the ET1 is steering towards right.

Task 2D

For the previous tasks evaluate the actions of the robot. Did they conform to your expectations? When driving back and forth, how far from the starting point did the robot end its movement?

In the previous task (task c) the action of the ET1 is **not as expected**. The ET1 doesn't drive in a straight line, **it steers towards right** both during the forward and backward direction. The ET1 is not **very far from the starting point**, since during the backward direction the ET1 mostly makes a circle than moving backward.

The **Right steering** is caused due to the **disproportional number of ticks** from both the motors. The Left wheel/Motor is faster compared to the Right Wheel/Motor.

Task 2E

Improve the accuracy of coming back to exactly the same point where you started use the rotary encoders on the wheels to measure how many revolutions each wheel travels. Implement two strategies:

- 1) Drive each motor for 5 complete wheel rotations, then reverse the motion
 - 2) Drive each motor for 10 seconds while monitoring the rotary encoders and slowing the faster moving motor to match the speed of the slower moving motor continuously.
- Comment on the performance of both.

Task 2E.1

Drive each motor for 5 complete wheel rotations, then reverse the motion

Objective:

To drive the ET1 forward 5 complete rotations of wheel and back ward.

Code Snippet:

Below is the code snippet used to drive the motor forward for 5 complete rotation and backward.

```
def wheel_rotation(PWM1, PWM2, num, dutycycle, direction):
    global left_sensor_tick_count, right_sensor_tick_count
    total_ticks = MOTOR_ENCODER_TICKS * num
    PWM1.ChangeDutyCycle(dutycycle)
    PWM2.ChangeDutyCycle(dutycycle)
    if direction == True:
        M1_forward()
        M2_forward()
    else:
        M1_backward()
        M2_backward()

    while((left_sensor_tick_count <= total_ticks) or (right_sensor_tick_count <= total_ticks)):
        print(f"Left ticks: {left_sensor_tick_count}, Right ticks: {right_sensor_tick_count} ")
        if right_sensor_tick_count == total_ticks:
            PWM2.ChangeDutyCycle(0)
            print("Right motor set to 0 duty")

            if left_sensor_tick_count == total_ticks:
                PWM1.ChangeDutyCycle(0)
                print("Left motor set to 0 duty")

            time.sleep(0.01)

        print_ticks(dutycycle, "Forward" if direction else "Backward")
        left_sensor_tick_count = 0
        right_sensor_tick_count = 0

while (1):
    if GPIO.input(Taster) == GPIO.LOW:
        wheel_rotation(PWM_1, PWM_2, 5, 60, True)
        print("forward done")
        time.sleep(1)
        wheel_rotation(PWM_1, PWM_2, 5, 60, False)
        print("Backword done")

        PWM_1.ChangeDutyCycle(0)
        PWM_2.ChangeDutyCycle(0)
```

Expected Result:

On pressing the push button,

- The ET1 should move forward in straight line for 5-wheel rotation.
- ET1 should drive backward in straight line for 5-wheel rotation.
- The ET1 should come back to starting position.

Actual Result:

- The ET1 is not moving forward in straight direction, it is steering towards right but it is completing 5-wheel rotation.
- The ET1 is not moving backward in straight line, it is steering towards right but it is completing 5-wheel rotation.
- ET1 doesn't come back to starting position.

Observation:

Below is the output from the code

<pre>pi@raspberrypi:~/work/task2 \$ python3 task2_e1.py Left ticks: 3, Right ticks: 0 Left ticks: 3, Right ticks: 0 : : Left ticks: 99, Right ticks: 84 Left ticks: 99, Right ticks: 85 Left ticks: 100, Right ticks: 85 Left motor set to 0 duty Left ticks: 101, Right ticks: 86 Left ticks: 101, Right ticks: 86 Left ticks: 102, Right ticks: 87 Left ticks: 102, Right ticks: 88 Left ticks: 103, Right ticks: 88 Left ticks: 103, Right ticks: 89 Left ticks: 104, Right ticks: 89 Left ticks: 104, Right ticks: 90 Left ticks: 105, Right ticks: 90 Left ticks: 105, Right ticks: 91 Left ticks: 106, Right ticks: 92 Left ticks: 106, Right ticks: 92 Left ticks: 107, Right ticks: 93 Left ticks: 107, Right ticks: 93 Left ticks: 107, Right ticks: 94 Left ticks: 108, Right ticks: 95 Left ticks: 108, Right ticks: 95 Left ticks: 109, Right ticks: 96 Left ticks: 109, Right ticks: 96 Left ticks: 109, Right ticks: 97 Left ticks: 110, Right ticks: 97 Left ticks: 110, Right ticks: 98 Left ticks: 110, Right ticks: 98 Left ticks: 111, Right ticks: 99 Left ticks: 111, Right ticks: 100 Right motor set to 0 duty Left ticks: 111, Right ticks: 100 Right motor set to 0 duty Forward : Dutycycle: 60 --> Left ticks: 112, Right ticks: 101 forward done</pre>	<pre>Left ticks: 4, Right ticks: 7 Left ticks: 4, Right ticks: 7 Left ticks: 5, Right ticks: 7 : : Left ticks: 99, Right ticks: 84 Left ticks: 99, Right ticks: 85 Left ticks: 100, Right ticks: 85 Left motor set to 0 duty Left ticks: 101, Right ticks: 86 Left ticks: 101, Right ticks: 86 Left ticks: 102, Right ticks: 87 Left ticks: 102, Right ticks: 88 Left ticks: 103, Right ticks: 88 Left ticks: 104, Right ticks: 89 Left ticks: 104, Right ticks: 89 Left ticks: 105, Right ticks: 90 Left ticks: 105, Right ticks: 90 Left ticks: 106, Right ticks: 91 Left ticks: 106, Right ticks: 91 Left ticks: 107, Right ticks: 92 Left ticks: 107, Right ticks: 92 Left ticks: 107, Right ticks: 93 Left ticks: 108, Right ticks: 94 Left ticks: 108, Right ticks: 94 Left ticks: 109, Right ticks: 95 Left ticks: 109, Right ticks: 95 Left ticks: 110, Right ticks: 96 Left ticks: 110, Right ticks: 96 Left ticks: 110, Right ticks: 97 Left ticks: 111, Right ticks: 97 Left ticks: 111, Right ticks: 98 Left ticks: 111, Right ticks: 98 Left ticks: 111, Right ticks: 99 Left ticks: 112, Right ticks: 99 Left ticks: 112, Right ticks: 100 Right motor set to 0 duty Left ticks: 112, Right ticks: 100 Right motor set to 0 duty Backward : Dutycycle: 60 --> Left ticks: 112, Right ticks: 101 Backward done</pre>
---	--

- Across both forward and backward drive, the left motor consistently registers higher tick counts than the right motor.

- Even though the cut of is set to 100 for both the wheels the left wheel is moving a more.

Task 2E.2

Drive each motor for 10 seconds while monitoring the rotary encoders and slowing the faster moving motor to match the speed of the slower moving motor continuously.

Objective:

To monitor the rotary encoders and matching the speed by slowing down the faster motor.

Theory:

We could continuously monitor the rotary encoders and use a PID controller to adjust the speed of the fast-moving motor.

Below is the table containing the data of the tick counts for different duty cycle for both direct duty cycle and duty cycle with PID corrections. We can observe that there is a considerable improvement in the difference between left and right tick counts with PID.

	Duty Cycle	Left Click	Right Click	Difference
Direct	30	206	182	24
	40	244	225	19
	50	267	251	16
	60	284	271	13
	70	296	286	10
	80	307	298	9
	90	314	306	8
With PID	30	174	175	-1
	40	232	231	1
	50	262	260	2
	60	281	279	2
	70	293	291	2
	80	302	299	3
	90	311	307	4

Code:

Below is the code snippet for the PID controller.

```
# PID control function to compute correction for motor speed based on tick difference
def pid_control(target_diff, actual_diff, integral, last_error, dt, kp, ki, kd):
    """
    Computes PID control output to minimize the difference between target and actual tick counts.

    Args:
        target_diff (float): Desired tick difference (setpoint, typically 0 for straight movement).
        actual_diff (float): Current tick difference (right - left sensor ticks).
        integral (float): Accumulated error for integral term.
```

last_error (float): Previous error for derivative term.

dt (float): Time step (seconds) since last update.

kp (float): Proportional gain.

ki (float): Integral gain.

kd (float): Derivative gain.

Returns:

tuple: (output, integral, error)

- output (float): PID correction to adjust motor speed.

- integral (float): Updated integral term.

- error (float): Current error for use as last_error in next call.

"""

Calculate error as the difference between target and actual tick difference

error = target_diff - actual_diff

Update integral term by accumulating error over time

*integral += error * dt*

Calculate derivative term as rate of change of error

derivative = (error - last_error) / dt if dt > 0 else 0

Compute PID output: proportional + integral + derivative

*output = kp * error + ki * integral + kd * derivative*

return output, integral, error

Core function to drive motors with PID control for straight movement

def PID_motor_drive(PWM1, PWM2, base_dutycycle, interval):

"""

Drives motors using PID control to maintain straight movement by balancing wheel ticks.

Args:

PWM1: PWM object for left motor.

PWM2: PWM object for right motor.

base_dutycycle (float): Base PWM duty cycle (0-100%) for motor speed.

interval (float): Duration (seconds) to drive the motors.

Uses global variables:

left_sensor_tick_count, right_sensor_tick_count: Encoder tick counts for left and right wheels.

"""

Access global tick counts for wheel encoders

global left_sensor_tick_count, right_sensor_tick_count

Record start time to track duration

start_time = time.time()

last_time = start_time

PID constants for proportional, integral, and derivative terms

KP = 5.0

KI = 0.1

KD = 0.01

Initialize PID variables

integral = 0.0

last_error = 0.0

```

# Run until the specified interval has elapsed
while (time.time() - start_time < interval):
    current_time = time.time()
    dt = current_time - last_time

    # Update every 0.02 seconds to avoid excessive computation
    if dt >= 0.02:
        # Calculate tick difference (right - left) to detect deviation
        tick_diff = right_sensor_tick_count - left_sensor_tick_count
        # Compute PID correction
        adjust, integral, last_error = pid_control(0, tick_diff, integral, last_error, dt, KP, KI, KD)
        # Adjust motor speeds based on tick difference
        if tick_diff < 0: # Right wheel lagging, slow left motor
            left_dutycycle = max(0, min(100, base_dutycycle - adjust))
            PWM1.ChangeDutyCycle(left_dutycycle)
        else: # Left wheel lagging or equal, adjust right motor
            right_dutycycle = max(0, min(100, base_dutycycle + adjust))
            PWM2.ChangeDutyCycle(right_dutycycle)

        # print(f"Backward - Left ticks: {left_sensor_tick_count}, Right ticks: {right_sensor_tick_count},
pid_adjust: {adjust:.4f}, "
        # f"Diff: {tick_diff}, Left PWM: {left_dutycycle:.1f}, Right PWM: {right_dutycycle:.1f}")
        # Update last_time for next iteration
        last_time = current_time
    # Small delay to prevent excessive CPU usage
    time.sleep(0.001)

# Stop motors by setting PWM duty cycles to 0
PWM1.ChangeDutyCycle(0)
PWM2.ChangeDutyCycle(0)

# Function to move the robot forward for a given interval
def move_forward(PWM1, PWM2, base_dutycycle, interval):
    """
    Moves the robot forward using PID control to maintain straight movement.

    Args:
        PWM1: PWM object for left motor.
        PWM2: PWM object for right motor.
        base_dutycycle (float): Base PWM duty cycle (0-100%) for motor speed.
        interval (float): Duration (seconds) to move forward.
    """
    # Set initial PWM duty cycles for both motors
    PWM1.ChangeDutyCycle(base_dutycycle)
    PWM2.ChangeDutyCycle(base_dutycycle)

    # Set motor directions to forward (assumes M1/M2_forward are defined externally)

```

```

M2_forward()
M1_forward()
# Drive motors with PID control
PID_motor_drive(PWM1, PWM2, base_dutycycle, interval)

# Function to move the robot backward for a given interval
def move_backward(PWM1, PWM2, base_dutycycle, interval):
    """
    Moves the robot backward using PID control to maintain straight movement.

    Args:
        PWM1: PWM object for left motor.
        PWM2: PWM object for right motor.
        base_dutycycle (float): Base PWM duty cycle (0-100%) for motor speed.
        interval (float): Duration (seconds) to move backward.
    """
    # Set initial PWM duty cycles for both motors
    PWM1.ChangeDutyCycle(base_dutycycle)
    PWM2.ChangeDutyCycle(base_dutycycle)

    # Set motor directions to backward (assumes M1/M2_backward are defined externally)
    M2_backward()
    M1_backward()

    # Drive motors with PID control
    PID_motor_drive(PWM1, PWM2, base_dutycycle, interval)

```

Expected Result:

On pressing the push button,

- The ET1 should move forward in straight line for 10 seconds.
- ET1 should drive backward in straight line for 10 seconds.
- The ET1 should come back to starting position.

Actual Result:

- The ET1 is almost moves forward in a straight line and steers a bit in some runs.
- The ET1 is almost moves backward in a straight line and steers a bit in some runs
- ET1 come back to starting position but with some difference.

Output:

```
pi@raspberrypi:~/work/task2 $ python3 task2_e2.py
enter the duty cycle : 50
initial right duty : 50, left duty: 50
Left ticks: 224, Right ticks: 226
forward done
initial right duty : 50, left duty: 50
Left ticks: 210, Right ticks: 208
Backward done
enter the duty cycle : 60
initial right duty : 60, left duty: 60
Left ticks: 245, Right ticks: 245
forward done
initial right duty : 60, left duty: 60
Left ticks: 243, Right ticks: 242
Backward done
```

Observation:

- The PID controller aims to balance wheel speeds for straight movement, but occasional tick mismatches cause the ET1 to slightly veer off course. Although the PID corrects the speed to resume straight travel, the robot continues along a parallel path rather than returning to its original straight-line trajectory.
- When returning to the starting position, the robot reaches the approximate location but not the exact point. The accuracy of the return depends on the extent of any steering deviations during forward or backward movement.

Task 2F

Use what you have learned in the previous tasks to create a program that will move the robot forward until it comes close to an obstacle (using the SR04 distance sensor to measure the distance), then turn right or left 90 degrees at random and continue moving until the next obstacle is encountered in the same fashion. Use the rotary encoders to turn exactly 90 degrees (as good as possible).

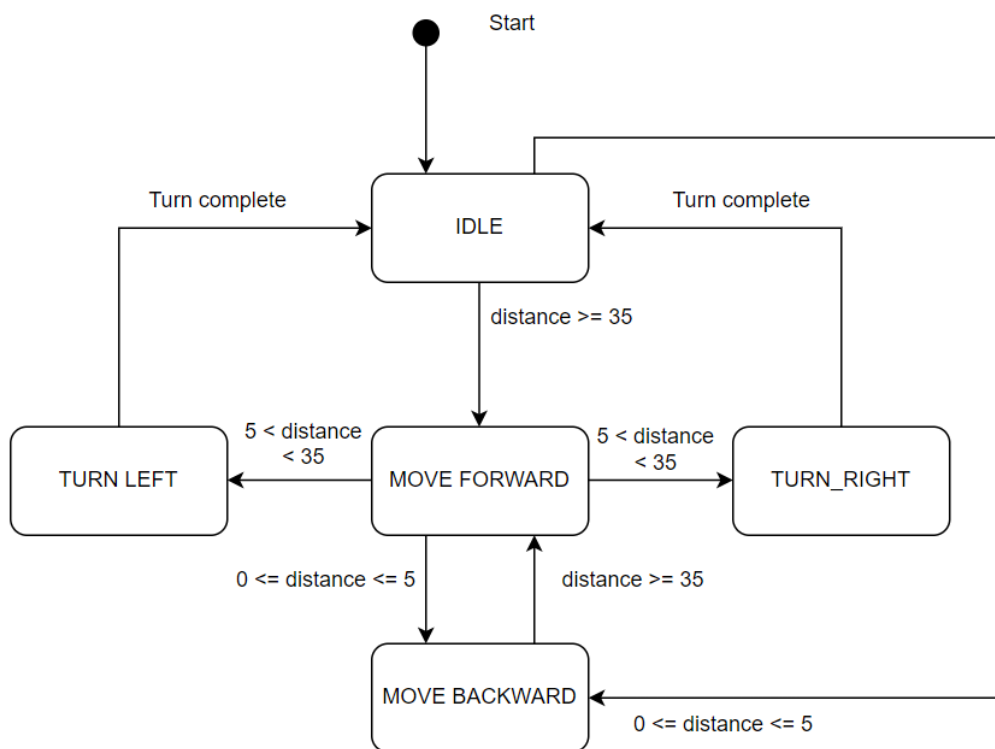
Objective:

To drive the ET1 with avoiding the obstacle using the distance data from the SR04.

Theory:

To detect the obstacle, we will be using the distance data from the SR04. To turn the ET1 right 90 degrees currently I found the approximate number of ticks which will turn right almost 90 degrees, I am using the hardcoded value now.

Below is the state diagram of the state machine used for the obstacle avoidance code.



Code:

Below is the code snippet used for object avoidance and turn:

```

# Function to turn the robot by a specified angle in a given direction
def turn(PWM1, PWM2, dutycycle, angle, direction):
    """
    Turns the robot by a specified angle (90, 180, or 360 degrees) in the given direction.

    Args:
        PWM1: PWM object for left motor.
        PWM2: PWM object for right motor.
        dutycycle (float): PWM duty cycle (0-100%) for the active motor during turn.
        angle (int): Desired turn angle (90, 180, or 360 degrees).
        direction (bool): True for right turn, False for left turn.

    Uses global variables:
        left_sensor_tick_count, right_sensor_tick_count: Encoder tick counts for wheels.
        et1_state: Robot state (updated to IDLE after turn).
    """
    global left_sensor_tick_count, right_sensor_tick_count, et1_state
    ticks = 0
    # Map angle to required encoder ticks (hardcoded values)
    if angle == 90:
        ticks = 14
    elif angle == 180:

```

```

    ticks = 30
elif angle == 360:
    ticks = 60
else:
    ticks = 0 # No turn if angle is invalid

if direction: # Turn right
    # Set left motor on, right motor off for right turn
    PWM1.ChangeDutyCycle(dutycycle)
    PWM2.ChangeDutyCycle(0)
    print("turn right")
    left_sensor_tick_count = 0 # Reset left tick count
    # Wait until left wheel reaches required ticks
    while left_sensor_tick_count <= ticks:
        time.sleep(0.01)
else: # Turn left
    # Set right motor on, left motor off for left turn
    PWM2.ChangeDutyCycle(dutycycle)
    PWM1.ChangeDutyCycle(0)
    right_sensor_tick_count = 0 # Reset right tick count
    # Wait until right wheel reaches required ticks
    while right_sensor_tick_count <= ticks:
        time.sleep(0.01)

# Stop both motors after turn
PWM1.ChangeDutyCycle(0)
PWM2.ChangeDutyCycle(0)
# Reset tick counts for both wheels
left_sensor_tick_count = 0
right_sensor_tick_count = 0
# Set robot state to IDLE after turn
et1_state = RobotState.IDLE

# Function to move forward with obstacle avoidance using a state machine
def move_forward_obstracle(PWM_1, PWM_2, duty, interval=20):
    """
    Moves the robot forward with obstacle avoidance, using a state machine to handle
    forward movement, backward movement, right turns, and stopping based on distance sensor.

    Args:
        PWM_1: PWM object for left motor.
        PWM_2: PWM object for right motor.
        duty (float): PWM duty cycle (0-100%) for motor speed.
        interval (float): Maximum runtime (seconds) for the function (default: 20).

    Uses global variables:
        et1_state: Robot state (e.g., MOVE_FORWARD, TURN_RIGHT, STOP).
        left_sensor_tick_count, right_sensor_tick_count: Encoder tick counts for wheels.

```

Dependencies:

DistanceSensor: Object for reading distance (assumed to be from a library like *gpiozero*).

move_forward, *move_backward*: External functions for PID-controlled movement.

RobotState: Enum defining robot states.

"""

```
global et1_state, left_sensor_tick_count, right_sensor_tick_count
# Initialize distance sensor (e.g., HC-SR04) with specified GPIO pins
distance_sensor = DistanceSensor(echo=27, trigger=25)
# Record start time to track runtime
runtime_start = time.time()
# Set initial state to MOVE_FORWARD
et1_state = RobotState.MOVE_FORWARD
# Run until the specified interval is reached
while time.time() - runtime_start < interval:
    # Read distance in centimeters (convert from meters)
    sensor_distance = distance_sensor.distance * 100
    # State transitions based on distance
    if 5 < sensor_distance < 35:
        # Obstacle detected within 5-35 cm, turn right
        et1_state = RobotState.TURN_RIGHT
    elif 0 <= sensor_distance <= 5:
        # Obstacle very close (0-5 cm), move backward
        et1_state = RobotState.MOVE_BACKWARD
    else:
        # No obstacle (distance > 35 cm), move forward if not already doing so
        if et1_state != RobotState.MOVE_FORWARD:
            et1_state = RobotState.MOVE_FORWARD

    # print(f"Robot state: {et1_state}")
    # Handle actions for each state
    if et1_state == RobotState.MOVE_FORWARD:
        # Move forward with PID control for 0.1 seconds
        move_forward(PWM_1, PWM_2, duty, 0.1)
    elif et1_state == RobotState.MOVE_BACKWARD:
        # Move backward with PID control for 0.1 seconds
        move_backward(PWM_1, PWM_2, duty, 0.1)
    elif et1_state == RobotState.TURN_RIGHT:
        # Stop motors briefly before turning
        PWM_1.ChangeDutyCycle(0)
        PWM_2.ChangeDutyCycle(0)
        time.sleep(0.2)
        # Perform a 90-degree right turn
        turn(PWM_1, PWM_2, duty, 90, True)
        time.sleep(1) # Pause after turn to stabilize
    elif et1_state == RobotState.IDLE:
        # Stop motors in IDLE state
        PWM_1.ChangeDutyCycle(0)
```



```

    PWM_2.ChangeDutyCycle(0)
    time.sleep(0.1)
elif et1_state == RobotState.STOP:
    # Stop motors and exit loop
    PWM_1.ChangeDutyCycle(0)
    PWM_2.ChangeDutyCycle(0)
    break
    # Small delay for state transitions and sensor readings
    time.sleep(0.01)

# Stop motors when exiting
PWM_1.ChangeDutyCycle(0)
PWM_2.ChangeDutyCycle(0)
# Set final state to STOP
et1_state = RobotState.STOP

```

Expected Result:

ET1 should avoid obstacle, on detection of obstacle turn 90 degrees right/left.

Actual Result:

ET1 avoids the obstacle, turns approximately 90 degrees continues to move.

Note: I have attached the video of the ET1 movement.

Observation:

Since the ultrasonic sensor is only at the front, if ET1 has an obstacle on the right or left side the distance cannot be detected and the ET1 will still move forward.

Task 2G

Make sure you close all open connections and pin assignments when closing the programs

Objective:

To cleanup all the opened connection and pin assignment.

Code:

Below is the code snippet for clean up

```

# Cleanup function
def cleanup():
    """
    Cleans up GPIO resources, stops PWM, and closes sensor connections.
    """

```

```

"""
global PWM_1, PWM_2, distance_sensor
print("Cleaning up GPIO and resources...")
try:
    # Close gpiozero devices first
    if distance_sensor:
        distance_sensor.close()
        distance_sensor = None
    # Stop PWM signals
    if PWM_1:
        PWM_1.stop()
    if PWM_2:
        PWM_2.stop()
    # Remove event detection for encoders
    try:
        GPIO.remove_event_detect(SENSOR_LEFT)
    except:
        pass # Ignore if already removed
    try:
        GPIO.remove_event_detect(SENSOR_RIGHT)
    except:
        pass # Ignore if already removed
    # Cleanup GPIO
    GPIO.cleanup()

    print("Cleanup complete.")
except Exception as e:
    print(f"Error during cleanup: {e}")

```

Output:

From the snapshot of the terminal, we can see that the on-user interrupt we have cleaned up by closing all connections.

```

pi@raspberrypi:~/work/task2 $ python3 task2_f.py
enter the duty cycle : 40
enter the interval : 10
^C
Program interrupted by user.
Cleaning up GPIO and resources...
Cleanup complete.
pi@raspberrypi:~/work/task2 $

```

Task 3

Your Task is to create a python program that will move the robot forward until it comes close to an obstacle and stop about 10cm from it. Instead of calling the python script from the shell it should be triggered from the index.html page using a checkbox. Review the preparation exercise and modify the webserver.js script to execute a python script instead of toggling the LED.

When the robot has stopped the website should indicate the distance from the obstacle as well as the distance travelled by each wheel and un-check the control box used to start the movement.

Using another control box (copy and rename the corresponding parts in the webserver and index files) you should now execute a second script that will move the robot backward 20cm and turn a random angle, then call the first script again to move until encountering an object.

Use a third control box it initiates a random movement by continuously initiating scripts one and two until the control box is unchecked (the current script will be finished then and no new one initiated) During the movement output the distance travelled by each wheel and the distance from the object stopped at on the webpage (one line per movement)

Summing up, firstly modify the index.html page with the html tags for displaying the distance and the checkboxes for initiating the movement to reflect the new desired behaviour.

Secondly modify the webserver.js file to change the behaviour of the checkboxes to run the python script that will trigger the movement

Thirdly create the necessary python scripts

Objective:

- i. Create a python script to move the robot forward till obstacle is at distance around 10cm from it.
- ii. Modify webserver.js to trigger or execute the python script from the checkbox in index.html for any changes.
- iii. The data from the robot such as distance from obstacle, distance traveled by each wheel (Number of encoder ticks) has to be displayed after the script stops.
- iv. Create a second python script to move robot backward by around 20cm ($10+20 = 30$ cm) and turn random angle.
- v. After the second script call the first script to move forward till obstacle.
- vi. Create another Checkbox in index.html, that will execute a second python script followed by first python script.
- vii. Create a Third Checkbox in index.html, The third check box when checked will do following:
 - o Execute or trigger the first and second python script to start the movement of robot.
 - o Stop movement if the checkbox is un-checked.
 - o Display the data from robot such as distance from obstacle and distance traveled by each wheel on webpage during each movement.

Implementation Details

Create a python script to move the robot forward till obstacle is at distance around 10cm from it.

Approach:

By using the knowledge from the previous tasks, we can divide this task into following:

1. Code a python script which will just move the robot forward as straight as possible. The code with PID controller as we used in task 2 could be used.

2. Read the distance from the SR04 sensors and detect the obstacle. The code for the distance detection from task 2 could be used. with some updates to the distance to stop and state changes.
3. Stop the rover around 10cm from obstacle. The code for the stopping the rover around stop distance of 10cm from obstacle could be achieved using the state machine used in task 2 with some updates to distance of stoppage and states.

Challenges faced:

- In the first task to move the robot forward, there were similar challenges as in task 2 and we used the same solution of using the PID controller to adjust the speed of the fast-moving motor.
- In task to stop the robot at 10 cm from the obstacle, there was the issue that robot continued to move forward and hit the obstacle mainly because the robot would take few ticks to completely come to stop because of the inertia.

Proposed Solution:

- Slow down the speed of the robot as the distance from the obstacle decreases, starting from a specific distance.

```
elif et1_state == RobotState.SLOWDOWN:
    slow_duty = max(20,int((sensor_distance / 100) * duty))
    move_forward(PWM_1, PWM_2, slow_duty, 0.1)
```

- Add a buffer to the Stop distance i.e 10% of duty cycle.

```
if sensor_distance <= (STOP_DISTANCE + max(10, int(STOP_BUFFER * duty))):
    et1_state = RobotState.STOP
elif STOP_DISTANCE < sensor_distance < (STOP_DISTANCE + max(20,
int(SLOWDOWN_BUFFER * duty))):
    et1_state = RobotState.SLOWDOWN
```

- But Challenge with this approach is some time the motor stops rotating below 25% duty cycle, then the robot stops way before the actual stop distance.

Modify webserver.js to trigger or execute the python script from the checkbox in index.html for any changes.

Approach:

- Before triggering the python script using the checkbox, the client and server relation has to be understood as well as importance of file such as webserver.js and index.html.
- Update webserver.js to execute move_till_obstacle.py when the checkbox in index.html is toggled, replacing LED toggling functionality.

- Use Socket.IO to handle checkbox events and spawn Python scripts via `child_process`.

Proposed Solution:

Modified `webserver.js` to include a `runPythonScript` function that spawns python script on the forward event received from the webpage.

```
function runPythonScript(socket, ldata, statusKey, scriptName, emitEvent,
direction) {
  /**
   * Run a Python script and emit parsed JSON data to the client.
   *
   * @param {Object} socket - Socket.IO socket for client communication.
   * @param {number} ldata - Checkbox state (0 or 1).
   * @param {string} statusKey - Key for status in JSON (e.g.,
'forwardStatus').
   * @param {string} scriptName - Python script to run (e.g.,
'move_till_obstacle.py').
   * @param {string} emitEvent - Socket event to emit (e.g., 'forwarddata').
   * @param {string} direction - Movement direction ('Forward' or 'Backward').
   * @returns {Promise<number>} Resolves with the script's exit code.
   */
  return new Promise((resolve, reject) => {
    const statusValue = ldata; // Checkbox state
    const pythonProcess = spawn('python3', [scriptName]); //Running Python
script
    let buffer = ''; // Accumulate stdout data
    // Process the data from the python to be sent back to client
    pythonProcess.stdout.on('data', (data) => {
      try {
        const jsonMatch = buffer.match(/\{.*?\}/); // Match first JSON object
        if (jsonMatch) {
          const parsedData = JSON.parse(jsonMatch[0]);
          parsedData[statusKey] = statusValue; // Add status of checkbox
          parsedData.distance = parsedData.distance.toFixed(2); // Format
distance to 2 decimals
          parsedData.direction = direction; // Add direction of movement
          socket.emit(emitEvent, parsedData); // Emit to client
          buffer = buffer.replace(jsonMatch[0], ''); // Clear processed JSON
        }
      } catch (error) {
        console.error(`JSON parse error (${scriptName}):`, error.message);
      }
    });

    pythonProcess.stderr.on('data', (error) => {
      console.error(`Python error (${scriptName}):`, error.toString());
    });
  });
}
```

```
pythonProcess.on('close', (code) => {
  console.log(`${scriptName} exited with code ${code}`);
  resolve(code);
});

pythonProcess.on('error', (err) => {
  console.error(`Failed to start ${scriptName}:`, err.message);
  reject(err);
});
});
}
```

Used socket.on('forward', ...) to capture checkbox state and emit forwarddata with parsed JSON.

```
socket.on('forward', async (ldata) => {
  // Run forward movement script
  console.log(`Received forward ldata: ${ldata}`);
  try {
    await runPythonScript(socket, ldata, 'forwardStatus',
      'move_till_obstacle.py', 'forwarddata', 'Forward');
  } catch (error) {
    console.error('Error in forward script:', error.message);
  }
});
```

Ensured non-blocking execution with Promises and error handling for script failures.

The data from the robot such as distance from obstacle, distance traveled by each wheel (Number of encoder ticks) has to be displayed after the script stops.

Approach:

To display the distance to the obstacle and wheel travel distances (encoder ticks) on index.html, we need to add the html tags for each.

We need to receive these data from the webserver and parse the received data.

The parsed data then has to be displayed in webpage.

In the webserver.js we have to process the output from the python script execution and form a JSON data to send it to client (index.html).

Proposed solution:

- In webserver.js, runPythonScript function parses JSON using regex (`/\{.*?\}/`) and add Status and direction then send it to the client. The code is shown in (ii).

- In index.html, socket.on('forwarddata', ...) updates <div> elements (fwd_distance, fwd_leftTicks, fwd_rightTicks) with data.distance, data.left_ticks, and data.right_ticks.

```
<div>
  <p><input type="checkbox" id="moveforward">Move Forward</p>
  <div id = "fwd_distance"></div>
  <div id = "fwd_leftTicks"></div>
  <div id = "fwd_rightTicks"></div>
</div>
```

```
socket.on('forwarddata', (data) => {
  console.log('Received forwarddata:', data);
  document.getElementById('fwd_distance').innerHTML = `Distance:
${data.distance || 'N/A'}`;
  document.getElementById('fwd_leftTicks').innerHTML = `Left Ticks:
${data.left_ticks || 'N/A'}`;
  document.getElementById('fwd_rightTicks').innerHTML = `Right Ticks:
${data.right_ticks || 'N/A'}`;
  document.getElementById('moveforward').checked = !!data.forwardStatus;
});
```

Create a second python script to move robot backward by around 20cm and turn random angle.

Approach:

Similar to (i) a python script is created, add the moving backward and turn random angle.

The robot has to move back 20 cm from its position and turn at a random angle. Similar behaviours were implemented in previous tasks, but slight updates may be required in state changes and stop distance.

Challenge Faced:

- As in (i) to move the robot backward, there were similar challenges as in task 2 and we used the same solution of using the PID controller to adjust the speed of the fast-moving motor.
- To stop the robot 20cm from starting point, there was the issue that robot used to still move backward mainly because the robot would take few ticks to completely come to stop because of the inertia.
- Turning random angle is calculated by considering the parameters such as wheel diameter, distance between wheels, number of ticks in one rotation etc., but even then, the angles are not exact but approximately as expected.

Proposed solution:

- Add a buffer to the Stop distance i.e 10% of duty cycle to try to stop around 20cm.

```
if sensor_distance <= (STOP_DISTANCE - min(10, int(STOP_BUFFER * duty))):
    et1_state = RobotState.MOVE_BACKWARD
```

```

        elif (STOP_DISTANCE - min(10, int(STOP_BUFFER * duty))) <
sensor_distance < (STOP_DISTANCE + min(10, int(STOP_BUFFER * duty))):
        et1_state = RobotState.TURN_ANGLE

```

- A function angle_to_ticks to calculate the number of ticks for the angle is created.

```

def angle_to_ticks(angle, wheel_diameter=6.5, ticks_per_rotation=20,
track_width=10.0):
    """
    Calculate encoder ticks required for a pivot turn.

    Args:
        angle (float): Turn angle in degrees.
        wheel_diameter (float): Wheel diameter in cm (default: 6.5).
        ticks_per_rotation (int): Encoder ticks per wheel rotation (default:
20).
        track_width (float): Distance between wheels in cm (default: 10.0).

    Returns:
        int: Number of ticks required for the active wheel.
    """
    turn_radius = track_width
    arc_length = (angle / 360) * 2 * math.pi * turn_radius # Arc length in cm
    wheel_circumference = math.pi * wheel_diameter # Wheel circumference in
cm
    rotations = arc_length / wheel_circumference
    ticks = int(round(rotations * ticks_per_rotation))
    return ticks

```

After the second script call the first script to move forward till obstacle.

Approach:

After second python script completes, automatically run first python script to move forward until around 10 cm from an obstacle.

Proposed Solutions:

- In webserver.js, the backward handler runs second python script (move_backward.py) followed by first python script (move_till_obstacle.py) using await runPythonScript function.

```

socket.on('backward', async (ldata) => {
    // Run backward movement script, followed by forward
    try {
        await runPythonScript(socket, ldata, 'backwardStatus',
'move_backward.py', 'backwarddata', 'Backward');
        await runPythonScript(socket, ldata, 'backwardStatus',
'move_till_obstacle.py', 'backwarddata', 'Forward');
    } catch (error) {

```



```

        console.error('Error in backward scripts:', error.message);
    }
});

```

- Ensured sequential execution with Promises, maintaining UI updates via Socket.IO.

Create another Checkbox in index.html, that will execute a second python script followed by first python script.

Approach:

Create a new check box to handle the second python script, could be achieved by Copying and renaming the checkbox logic required consistent event naming and UI updates.

Proposed Solutions:

- Added `<input type="checkbox" id="movebackward">` in index.html, emitting the backward event on change.

```

<p><input type="checkbox" id="movebackward"> Move Backward</p>
.
.
.
backwardBox.addEventListener('change', () => {
    console.log('Backward checkbox changed');
    socket.emit('backward', Number(backwardBox.checked));
});

```

- Implemented `socket.on('backward', ...)` in webserver.js to run both scripts sequentially.

Create a Third Checkbox in index.html, The third check box when checked will do following:

- **Execute or trigger the first and second python script to start the movement of robot.**
- **Stop movement if the checkbox is un-checked.**
- **Display the data from robot such as distance from obstacle and distance traveled by each wheel on webpage during each movement.**

Approach:

- We have to create a third check box similar to (vi).
- The third checkbox should continuously trigger the first and second script continuously when checked. Could be achieved by using similar logic as (v)

- Stop when unchecked finishing the current script. Could be achieved by tracking the checkbox value at the webserver.js.
- Display distance and wheel ticks for each movement in a table. Could be achieved by creating a table to display the data in index.html and sending the data from the webserver after each script execution.

Proposed Solution:

In webserver.js, socket.on('random', ...) sets randomvalue = ldata and calls runRandomScripts, which loops scripts while randomvalue === 1. Used isRunningRandom flag to prevent concurrent loops. Ensured stopping by checking randomvalue after each script, finishing the current script if unchecked.

```
socket.on('random', (ldata) => {
  // Toggle random mode and run scripts if enabled
  randomvalue = ldata;
  if (randomvalue === 1) {
    runRandomScripts(socket);
  }
});
```

```
async function runRandomScripts(socket) {
  /**
   * Run forward and backward Python scripts in a loop until randomvalue is 0.
   *
   * @param {Object} socket - Socket.IO socket for client communication.
   */
  if (isRunningRandom) {
    console.log('Random scripts already running, skipping');
    return;
  }
  isRunningRandom = true;

  try {
    while (randomvalue === 1){
      // Run forward movement
      await runPythonScript(socket, randomvalue, 'randomStatus',
        'move_till_obstacle.py', 'randomdata', 'Forward');
      // Run backward movement
      await runPythonScript(socket, randomvalue, 'randomStatus',
        'move_backward.py', 'randomdata', 'Backward');
    }
  } catch (error) {
    console.error('Error in random scripts:', error.message);
  } finally {
    isRunningRandom = false;
    console.log('Random scripts stopped');
  }
}
```

```
}
```

In index.html, `socket.on('randomdata', ...)` calls `addTableRow(data, 'rnd')` to add rows to `rndDataTable` with Direction, Distance, Left Tick, and Right Tick.

```
// Add a row to the specified table
function addTableRow(data, prefix) {
  /**
   * Append a row to the table with movement data.
   *
   * @param {Object} data - Data object with direction, distance,
left_ticks, right_ticks.
   * @param {string} prefix - Table prefix ('bkwd' or 'rnd').
   */
  const table = document.getElementById(`${prefix}DataTable`);
  const tbody = document.getElementById(`${prefix}DataBody`);

  // Show table on first data
  if (!table.classList.contains('visible')) {
    table.classList.add('visible');
  }

  const row = document.createElement('tr');
  row.innerHTML = `
    <td>${data.direction}</td>
    <td>${data.distance || 'N/A'}</td>
    <td>${data.left_ticks || 'N/A'}</td>
    <td>${data.right_ticks || 'N/A'}</td>
  `;
  tbody.appendChild(row);
}
```

```
// Handle random movement data
socket.on('randomdata', (data) => {
  console.log('Received randomdata:', data);
  addTableRow(data, 'rnd');
});
```

Results:

- When the Checkbox 1 is checked or unchecked, the robot moves forward and stops around 10cm from the obstacle. The data such as distance from obstacle, number of ticks from each wheel is displayed on the webpage.

Control ET1

☒ Move Forward

Distance: 11.32

Left Ticks: 92

Right Ticks: 91

☐ Move Backward

☐ Move Random

- When the Checkbox 2 is checked or unchecked, the robot moves back by around 20cm and then turns a random angle between 0 to 360, then the first script is called to move the robot till it finds an obstacle.

☒ Move Backward

Direction	Distance	Left Tick	Right Tick
Backward	28.35	23	18
Forward	5.42	100	96

- When checkbox 3 is checked, the robot continuously executes the first and second python script for movement. This movement is stopped when we uncheck the checkbox.

Task 4

The task involves integrating four different programs to achieve coordinated robot movement and real-time sensor data visualization. These include a C program for ultrasonic distance sensing, two Python scripts (one for sensors and fusion logic, another for motor control), and a Node.js webserver for user interaction. The robot should execute a predefined sequence of movements while broadcasting fusion data to the UI.

Objective:

- Read SR04 ultrasonic sensor data using a C program and send it to a Python script.
- Use a Python sensor server to receive distance, read line sensors, compute fusion data, and communicate with a motor controller.

- iii. Control the robot's movement using a separate Python motor controller, which utilizes encoder ticks and PID logic.
- iv. Start the behavior from a checkbox in the web interface and show live fusion data.
- v. Demonstrate repeatable movement and return to initial position.
- vi. Display fusion data (speed, turning ticks, delta distance) in a live-updated web table.

Implementation Details

C Program: sr04_client.c

Approach:

- The wiringPi library was used to measure echo time and compute distance.
- Established a TCP client to connect to the Python server and send distance every 50ms.

Proposed Solution:

- Send formatted distance string via TCP socket.
- Receive and print line sensor feedback to debug.

Python Sensor Server: soc_server.py

Approach:

- Listens to TCP from sr04_client and UNIX socket from motor controller.
- Receives and parses distance and line data.
- Maintains tick counts and calculates speed, turning, and distance delta.
- Sends fusion data via TCP to the Node.js webserver.

Challenges Faced:

- Managing concurrent socket connections and race conditions.

Proposed Solution:

- Used multithreading for listening, computing, and process monitoring.
- Sockets were handled in the threads.

Python Motor Controller: motor_task4.py

Approach:

- Used encoder interrupts to count wheel ticks.
- Implemented PID control to synchronize wheel speeds.

- Programmed predefined motion: forward 40 cm, turn 30°, forward 20 cm, reverse, and repeat.

Proposed Solution:

- Get the distance value from the C program using a TCP socket connection, parse it in JSON format, and use it to evaluate proximity to obstacles in real-time.
- Used distance-to-ticks and angle-to-ticks conversions.
- Stopped based on SR04 reading and average encoder ticks.

Node.js Webserver and UI Integration

Approach:

- Modified webserver.js to listen for a checkbox event and launch soc_server.py.
- Established TCP server to receive and forward fusion data to the browser via Socket.IO.
- Displayed fusion data in an HTML table using JavaScript updates every 100 ms.

Proposed Solution:

- Leveraged Socket.IO for real-time bi-directional data flow.
- JavaScript buffers and updates table with speed, turning ticks, and delta distance.

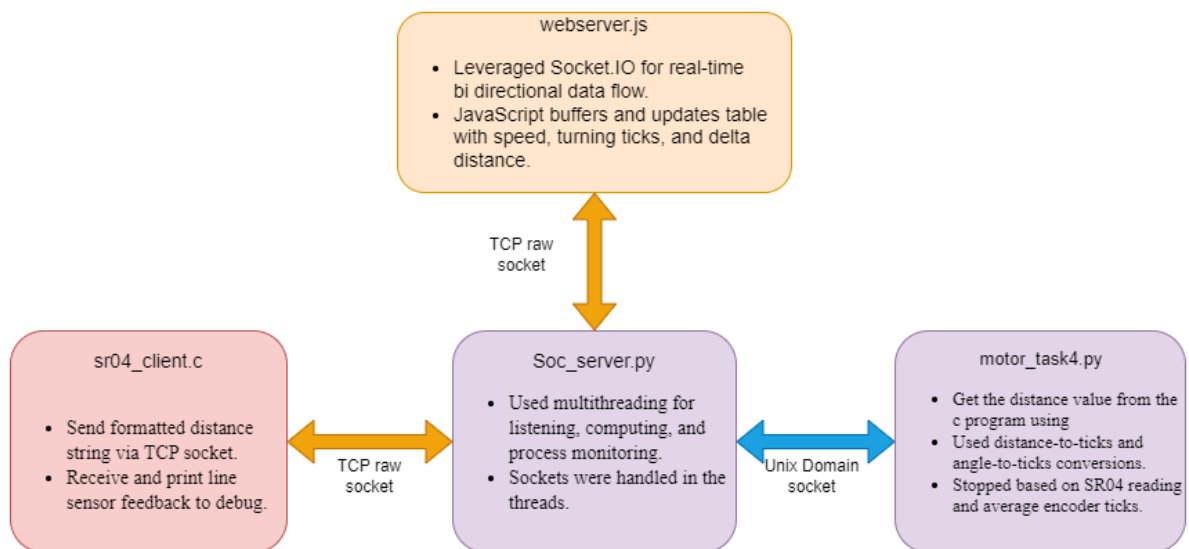


Figure 6: The Data flow between the processes

Results:

The robot successfully executed the predefined movement sequence:

- Moved forward 40 cm toward the wall

- Turned 30° to the right and moved 20 cm forward
- Reversed 20 cm and turned back to original orientation
- Repeated the above steps for the left side

The live fusion data displayed on the webpage showed real-time speed (in cm/s), turning rate (in encoder ticks), and delta distance (in cm). The speed varied based on the difference in wheel encoder ticks, and turning was accurately reflected in the turning tick data.

The webpage effectively displayed all sensor feedback and robot state, confirming proper communication between all four modules. No packet loss or timing delays were observed during movement cycles.

The screenshot below displays sensor details captured during movement.

Control ET1

☒ Sensor Fusion

Speed: N/A cm/s
Turn: N/A ticks
Delta Distance: 68.76 cm

Fusion Data Log

Time	Speed (cm/s)	Turn (ticks)	Distance (cm)
14:23:51.066	0	0	0
14:23:51.119	0	0	0
14:23:51.170	0	0	0
14:23:51.221	0	0	0
14:23:51.273	0	0	0
14:23:51.324	0	0	0
14:23:51.375	0	0	0
14:23:51.427	0	0	0